StepLens: Visual Analytics for Diagnosing Error Detection Models in Procedural Cooking Videos

Team 17 · Harvey Yang (cy2941@nyu.edu) · Jeremy Hsu (wh2757@nyu.edu) · DS-GA 3001, Fall 2026

1. Overview

We will build StepLens, an interactive visual analytics system that shows *where and why* video models fail to detect mistakes in cooking videos. CaptainCook4D [1] provides 384 egocentric recordings (94.5 hours) with step-level error labels in 7 categories, so the ground truth for every miss is known. The paper reports only aggregate numbers (the best supervised model on the step split reaches 75.7 AUC and 55.4 F1), which hide which error types, recipes, steps, people, and kitchens the models miss. StepLens links four views, an error-category × model performance matrix, recipe task graphs colored by miss rate, recording timelines that align ground-truth errors with per-second model scores and the video, and a slice table that ranks metadata subgroups by error rate, and uses them to compare backbones (Omnivore, SlowFast, multimodal ImageBind). We will check that the failure modes it surfaces hold on held-out data, across models, and across training seeds, and evaluate the tool through case-study walkthroughs with video evidence.

2. Key paper and critical analysis

Paper. R. Peddi et al., *CaptainCook4D: A Dataset for Understanding Errors in Procedural Activities*, NeurIPS 2024 Datasets and Benchmarks Track [1].

What it does. The paper releases an egocentric 4D dataset of people following recipes, where some recordings contain intentional mistakes. Eight participants recorded 384 videos (94.5 hours) of 24 WikiHow recipes in 10 kitchens, wearing a HoloLens 2 (RGB, depth, IMU, head and hand pose, audio) and a head-mounted GoPro (4K, 30 fps). There are 5.3K step annotations with start and end times, 10K fine-grained action annotations for 20% of the data, and a task graph (a DAG of step-order constraints) per recipe. Every step is labeled with errors from 7 categories, and a step can carry several errors. The released files also have an eighth catch-all column, *Other*, which the paper does not report; we show it separately and exclude it from category-level claims. Counts read from the paper's Figure 3 (approximate): Order 795, Technique 502, Preparation 410, Measurement 331, Missing step 285, Timing 177, Temperature 66.

Benchmarks and results. The paper defines supervised error recognition, zero-shot error recognition with video-language models, multi-step localization, and procedure learning. For supervised error recognition it splits each video into 1-second segments, extracts features with frozen backbones (3D-ResNet, SlowFast, X3D, Omnivore, ImageBind), and trains an MLP or transformer head that labels each step as error or normal, under six data splits (environment, person, recipe, recording, step, recording type). The best step-split result is Omnivore with a transformer head: 55.39 F1, 75.7 AUC, 69.96 accuracy. On the harder recording split, four-modality ImageBind (video, audio, depth, text) reaches 56.02 F1 and 65.25 AUC. Zero-shot video-language models do much worse: per-category F1 for Technique, Temperature, and Timing errors is between 0.6 and 5.1. Localization models trained on normal videos also drop sharply on error videos (Omnivore features, environment split: mAP@tIoU 0.1 falls from 34.65 to 12.51).

Main contribution and generality. The main contribution is data: deliberately induced errors labeled at the step level with a category and a free-text description, where most egocentric datasets (Ego4D, EPIC-KITCHENS) record only correct behavior. The collection protocol transfers to other procedural domains (assembly, lab or medical procedures), but the benchmark results are narrower: one domain, 24 recipes, 8 participants, one geographic area, and a Persons split that holds out only 2 people.

Flaws we found by reading the paper and the released code.

- Results are reported only as aggregate accuracy, F1, and AUC. Supervised models get no per-category, per-recipe, or per-step breakdown, so we cannot tell which errors are actually caught.
- The supervised benchmark never scores Missing Step errors. Skipped steps are annotated with start and end time −1 and the released dataloader drops them, so one of the seven categories (about 285 of roughly 2,566 errors) is outside the benchmark entirely.
- The released step split is not random. The split code shuffles a temporary list but then slices steps in annotation order, so training takes the first 75% of each recording's steps, the phase it calls "test" the next 16%, and "val" the last 9%. The three phases are different populations (mean relative position in the recipe 0.35, 0.78 and 0.91), which biases any analysis of

position in the recipe and makes model selection on "val" unreliable for "test". We add a seeded random step split with the same proportions as a control.

- Decision thresholds are not fully documented. The README states fixed thresholds of 0.6 (step split) and 0.4 (recording split), but the released evaluation code only sweeps 0.5 and 0.6 and never uses 0.4. On imbalanced labels this matters: Omnivore with an MLP head scores 71.09 accuracy but only 24.26 F1.
- Error frequencies follow the authors' scripts rather than natural behavior, and the categories are imbalanced (Order 795 vs. Temperature 66). The tables also have small inconsistencies: Omnivore MLP and transformer heads report the identical 75.7 AUC, and in Table 8 the TimeChat Missing Steps row reports precision 2.2 and recall 0.4 but F1 12.4 (the formula gives about 0.7; 12.4 equals that model's Preparation F1, suggesting a copy error).

What we take from it. We reuse the dataset, annotations, step and recording splits, released features, and the supervised error-recognition pipeline as our ML baseline, and we adopt the 7-category taxonomy as the main axis of analysis. A neat detail we build on: the paper evaluates localization models trained only on normal videos against error videos, showing that models break exactly where people deviate from the procedure. The questions the paper leaves open motivate our project: are failures concentrated in specific categories or steps, do different backbones miss the same errors, and do failures correlate with person, kitchen, or step duration?

3. Proposed project and innovation

Research questions. Each is answerable with evidence from the labeled errors.

- **RQ1: Where do error-recognition models fail?** Which error categories, recipes, steps, and conditions (person, kitchen, step length, position in the recipe) account for most missed errors and false alarms, and are these failures systematic or random?
- **RQ2: How do models differ?** Do Omnivore, SlowFast, and multimodal ImageBind miss the same errors, or do they complement each other?

How visualization and ML connect. The ML layer trains the paper's error-recognition models and stores a prediction and per-second score for every step; the visualization layer joins those predictions with the ground-truth labels, the recipe task graphs, the metadata, and the video, so that a person can find, explain, and verify systematic failures that aggregate metrics cannot show. Findings from the views feed back into the ML layer as targeted model changes.

Approach. StepLens has four linked views, so a selection in one filters the others: (1) a *performance matrix* with error categories as rows and models as columns, showing recall and false-alarm rate, with a threshold slider that replaces the paper's fixed 0.6 and 0.4 thresholds; (2) a *task-graph view* that draws each recipe's DAG with steps colored by miss rate, to show whether failures cluster at particular stages such as measuring or heating; (3) a *timeline and video view* for one recording, with ground-truth step segments and error labels above each model's per-second scores, where clicking a step plays that clip of the 360p GoPro video; and (4) a *slice table* that enumerates metadata slices (for example, one error category for one person in one kitchen) and ranks them by error rate and lift over the average, with a minimum slice size. A *compare mode* shows errors caught by one model but missed by another (RQ2). As a stretch goal we will add a UMAP projection of step features colored by correct versus missed.

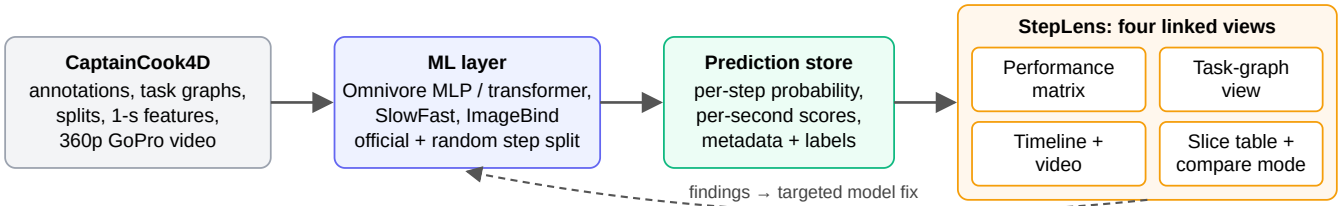


Figure 1. StepLens pipeline: data, models, a prediction store, and four linked views; findings feed back into the ML layer.

How it differs from existing work. In the visual analytics for deep learning surveyed by Hohman et al. [8], model-diagnosis tools such as ModelTracker [9], Squares [10], Manifold [11], the What-If Tool [12], Calibrate [13] (now the PyCalibrate library from NYU VIDA), and Domino [14] target static classification on images or tabular data. StepLens borrows per-class matrices, model comparison, and slice discovery from that line of work and adds what video procedures need: steps aligned in time, recipe task graphs, and linked video playback, for errors where the order and timing of steps are part of the label. Procedural-video datasets (Assembly101 [2], HoloAssist [3], PREGO [4], HD-EPIC [5], Ego4D and Ego-Exo4D [6, 7]) all evaluate with aggregate metrics and none provides a tool for inspecting failures.

Expected contributions. (1) **StepLens**, an open-source visual analytics system for step-level error-detection models. (2) **A failure analysis of the paper's baselines** broken down by category and slice, the breakdown the paper does not report, with a corrected random step split. (3) **A case-study evaluation** showing which failure modes the linked views surface that the paper's tables cannot, and whether those findings survive hold-out, model-switch, and seed checks. If time allows, (4) **a model improvement guided by the findings**, such as category-aware thresholds or reweighting of under-detected categories, measured against baseline F1 and AUC; this is a should-have item (Section 4) and the first three contributions stand without it.

4. Project plan and evaluation

Data. Everything we need is public and we have checked each source. The project website, downloader, and error-recognition code are Apache 2.0; the annotations repository's README says Apache 2.0 while its LICENSE file is MIT, and we cite it as Apache 2.0 with a note. The feature archive on Box downloads without a login; it holds 114 GB, but the 1-second segment features we need are 9 GB (the 105 GB of frame features are not needed). The archive has no depth features, so we reproduce the three-modality ImageBind model (video, audio, text) rather than the four-modality one. We do not need raw 4K or 3D sensor data (several terabytes).

What we use	Source	Use in project
Step and error annotations, task graphs, data splits (CSV, JSON)	github.com/CaptainCook4D/annotations	Ground truth, category labels, slices (person, environment)
Precomputed 1-s segment features: Omnivore, SlowFast, 3D-ResNet, X3D; ImageBind video, audio, text at 2 s	Box archive linked from github.com/CaptainCook4D/feature_extractors	Train and evaluate models without raw video
Baseline training code (shell script per backbone, MLP and transformer heads)	github.com/CaptainCook4D/error_recognition	Reproduce the paper's baselines
GoPro video at 360p (MP4)	github.com/CaptainCook4D/downloader via Box	Video playback in the timeline view

About 60 of the 384 recordings have only GoPro video and no HoloLens data, about 40 more lost the spatial and IMU streams, and ImageBind features exist for 383 recordings (12_6 is absent); we report which recordings each model covers.

Implementation. *ML layer:* we train the paper's supervised models with the released features and code: Omnivore with an MLP head and with a transformer head, SlowFast, and three-modality ImageBind, each on the step and recording splits, with the official recipe (learning rate 1e-3 for the MLP head, 1e-6 for the transformer head, model selection by validation AUC) and with a tuned learning rate, and also on our random step split. For every step we store the predicted error probability, the per-second scores, and metadata (recipe, step, error categories, person, kitchen, step length, position in the recipe). Heads are small and features are frozen, so training fits on one GPU (NYU HPC or Colab). *Visualization layer:* a Python backend (PyTorch, pandas, FastAPI) serving a D3.js front end with the four linked views above.

Evaluation. We evaluate at three levels: the models, the findings, and the tool.

Level	What we measure	Baseline	Success looks like
Models (sanity check)	Accuracy, F1, AUC on the step and recording splits	Paper's Table 2, e.g. Omnivore transformer head at 75.7 AUC on the step split (our ImageBind lacks depth, so we expect a gap there and report it)	Reproduction within about 2 points
Findings (RQ1, RQ2)	Per-category recall, false-alarm rate, slice error rate, model agreement (overlap of caught errors)	Aggregate metrics only, as in the paper	Failure slices found on the validation part of our random step split also hold on its test part; official-split results reported alongside
Tool	Case studies: whether each linked view surfaces a failure mode the aggregate tables cannot, and whether the finding survives hold-out, model-switch, and seed checks	The paper's aggregate tables	Each case study yields a confirmed, reproducible finding with a shareable view link and video evidence
Model fix (should-have)	F1 and AUC after a change guided by the findings (slice-aware threshold calibration, per-recording feature centering, or reweighting of under-detected categories)	Unchanged baseline model	Higher test F1, with gains in the targeted slices or categories

Because the official "val" and "test" phases are different populations (Section 2), slice discovery and confirmation use our seeded random step split; we report bootstrap confidence intervals and require a minimum slice size. We will write three or four case-study walkthroughs, for example whether supervised models also miss the Temperature and Timing errors that zero-shot models almost never detect, and whether a slice flagged for one model is a property of the data or of that run, with video clips as evidence. We do not run a controlled user study; its materials (protocol, task sheet, questionnaire, reference answers, and a notebook baseline) are prepared as an appendix for future work.

Timeline. The mid-term update is due Nov 3, presentations are on Dec 1 or Dec 8, and the final report is due Dec 14, so we have six weeks to a presentable system and eight to the report. Work is split into must-have, should-have, and stretch items so the project degrades gracefully.

Weeks	Dates	Work	Owner
1–2	Oct 19 – Nov 1	Download features and annotations, confirm archive size and license, reproduce the Omnivore baselines on the step split, build the random step split	Harvey
3–4	Nov 2 – Nov 15	Mid-term update (Nov 3); reproduce ImageBind and SlowFast, store per-step predictions and metadata, build the performance matrix and slice table	Jeremy
5–6	Nov 16 – Nov 29	Timeline and video view, compare mode, task-graph view, failure analysis with validation-to-test confirmation, draft case studies	both
7	Nov 30 – Dec 6	Presentation (if scheduled Dec 1), seed robustness runs, model fix guided by the findings	Harvey
8	Dec 7 – Dec 13	Presentation (if scheduled Dec 8), finalize case studies, final report (due Dec 14)	both

Must-have: Omnivore (both heads) and ImageBind baselines, performance matrix, slice table, timeline and video view, compare mode, per-category and per-slice failure analysis (RQ1, RQ2). *Should-have:* SlowFast baseline, task-graph view, threshold-based model fix, multi-seed confidence intervals. *Stretch:* UMAP projection and reweighting-based model fixes.

Risks and fallbacks. If the feature archive is too large or does not download, we use Omnivore features only or extract them ourselves from the 360p video. If our reproduction lands more than 2 points from the paper, we report our own numbers, document the differences (thresholds, seeds, head configuration), and use our models for the analysis anyway. If video playback or hosting is a problem, we pre-cut a clip per step and serve them as static files.

AI use disclosure. As the course AI policy requires, we disclose that we used Claude (Anthropic) to brainstorm the topic, summarize the CaptainCook4D paper, check that the dataset and code are available, draft text, tables, and the system diagram, and fact-check the proposal against the paper and the released repositories.

References

1. R. Peddi et al. CaptainCook4D: A Dataset for Understanding Errors in Procedural Activities. NeurIPS 2024, Datasets and Benchmarks Track. Code and data: github.com/CaptainCook4D.
2. F. Sener et al. Assembly101: A Large-Scale Multi-View Video Dataset for Understanding Procedural Activities. CVPR 2022.
3. X. Wang et al. HoloAssist: An Egocentric Human Interaction Dataset for Interactive AI Assistants in the Real World. ICCV 2023.
4. A. Flaborea et al. PREGO: Online Mistake Detection in PRocedural EGOcentric Videos. CVPR 2024.
5. T. Perrett et al. HD-EPIC: A Highly-Detailed Egocentric Video Dataset. CVPR 2025.
6. K. Grauman et al. Ego4D: Around the World in 3,000 Hours of Egocentric Video. CVPR 2022.
7. K. Grauman et al. Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. CVPR 2024.
8. F. Hohman, M. Kahng, R. Pienta, D. H. Chau. Visual Analytics in Deep Learning: An Interrogative Survey for the Next Frontiers. IEEE TVCG 2019.
9. S. Amershi et al. ModelTracker: Redesigning Performance Analysis Tools for Machine Learning. CHI 2015.
10. D. Ren et al. Squares: Supporting Interactive Performance Analysis for Multiclass Classifiers. IEEE TVCG 2017.
11. J. Zhang et al. Manifold: A Model-Agnostic Framework for Interpretation and Diagnosis of Machine Learning Models. IEEE TVCG 2019.
12. J. Wexler et al. The What-If Tool: Interactive Probing of Machine Learning Models. IEEE TVCG 2020.
13. P. Xenopoulos et al. Calibrate: Interactive Analysis of Probabilistic Model Output. IEEE TVCG 2023 (IEEE VIS 2022).
14. S. Eyuboglu et al. Domino: Discovering Systematic Errors with Cross-Modal Embeddings. ICLR 2022.